

Apex Framework

Handbook

A practical guide to using the Apex Framework

Version 4.1.0

TABLE OF CONTENTS

Chapter	Title	Page
1	Introduction	3
2	Development Lifecycle	4
3	Practical Workflows	6
4	Use Cases	8
5	Customization	17
6	FAQs	19

Chapter 1: Introduction

Apex is a tool-agnostic AI development framework that gives your AI coding tool project awareness, consistency, and built-in quality gates. It works with Cursor, VS Code with GitHub Copilot, and Claude Code.

WHO THIS HANDBOOK IS FOR

This handbook is for anyone using the Apex Framework in their day to day work: developers, functional consultants, technical leads, and project managers. No prior experience with AI coding tools is assumed.

This handbook covers **how to use** the framework: the development lifecycle, practical workflows you will follow every day, and real use cases that show the framework in action from start to finish. It does not cover setup, tool installation, or approvals. For that, refer to the **Apex Framework Prerequisites** document, which walks you through everything you need before you start using the framework.

THE PROBLEM

- Every AI session starts from zero. No memory of your project's architecture, naming conventions, or standards.
- AI output is generic. Wrong patterns, wrong structure, significant rework before it can be used.
- No consistency across developers. Each person prompts differently, gets different results.
- No quality gates. Code goes from generated to accepted with nothing in between.

WHAT APEX DOES FOR YOU

- One-time setup gives every developer on the team the same project context from day one.
- Specialist agents handle different phases of development automatically, following your project's conventions.
- A structured lifecycle ensures code is planned, built, reviewed, and tested before it ships.
- Works across Cursor, VS Code with GitHub Copilot, and Claude Code. The framework stays the same regardless of which tool you use.

Chapter 2: Development Lifecycle

Every task in Apex follows a structured lifecycle. The framework handles the heavy lifting, but you are always in control. At every major decision point, the framework pauses and waits for you before moving forward. Nothing ships without your approval.



WHERE YOU STEP IN

The table below shows each phase, what the framework does, and exactly where you are involved. The "Your Role" column highlights every point where the framework pauses and waits for your input.

Phase	What Happens	Your Role (Human in the Loop)
Plan	You describe what you want to build. The framework scans your project, identifies reusable code, checks your standards, and produces a detailed implementation plan with file paths, approach, and open questions.	YOU APPROVE the plan before anything gets built. If something is wrong, you give feedback and the framework revises it. The framework will not proceed until you say yes.
Build	The framework implements the approved plan step by step, following your project's conventions, naming patterns, and code standards. It compiles and auto-fixes any build errors.	You see progress updates. For larger tasks, the framework PAUSES mid-build to confirm direction before continuing.
Review	A specialist reviewer scans the code for security vulnerabilities (CRITICAL/HIGH), logic errors, architecture violations, and standards compliance. Every finding includes file, line number, severity, and a specific fix recommendation.	YOU REVIEW the findings. You decide which fixes to accept. The reviewer never changes your code directly. A separate process applies fixes you approve.
Test	Unit tests are generated targeting 80% coverage, then executed. Failures are analyzed and fixed automatically. You see a summary of pass/fail results.	YOU REVIEW test results and coverage. If tests reveal unexpected behavior, you decide whether to adjust the implementation or the test expectations.
Deploy	The framework builds and deploys to your local environment. Logs are checked to confirm the new code is active and running.	YOU CONFIRM deployment. For larger tasks, the framework PAUSES and shows you what was built before deploying. It will not deploy without your go-ahead.
Validate	The framework validates the deployed output against the original requirements. For UI components, this includes visual comparison. For APIs, it tests endpoints. For services, it checks logs.	YOU REVIEW the validation report and confirm the output meets your expectations. This is the final quality gate before the task is marked complete.

Chapter 3: Practical Workflows

This chapter shows you how to work with the framework day to day. What you type, what to expect, and how the flow feels as a user. The pattern is always the same: you tell the framework what you want, it does the work, and you review the result.

HOW YOU INVOKE COMMANDS

The way you start a workflow depends on which AI coding tool you use. The commands and behavior are the same across all three tools. Only the invocation method differs:

- **Cursor:** Type the command directly in the chat panel. For example, type **/planner** followed by your request. Cursor supports slash commands, subagents, skills, hooks, and MCP servers natively.
- **VS Code with GitHub Copilot:** Open the Copilot chat panel and attach the command file using the # button. Select the command from the `.claude/commands/` folder (e.g., `planner.md`), then type your request in the chat. Copilot loads the command context from the attached file.
- **Claude Code:** Type the command directly in the terminal. For example, type **/planner** followed by your request. Claude Code supports slash commands, subagents, skills, hooks, and MCP servers natively.

Regardless of which tool you use, the framework behavior is identical. The same agents run, the same skills load, and the same lifecycle applies. Pick the tool your team uses and follow the invocation method above.

STARTING A NEW TASK

For any development task, start with **/planner** followed by what you want to do. The planner is the main entry point for all development work. It figures out the complexity, decides which agents and skills to use, and produces a structured plan for your approval.

- Example: **/planner build a hero banner component based on this Figma design**
- Example: **/planner fix the login redirect issue on the dashboard page**
- Example: **/planner create a REST API endpoint for user preferences**

After you invoke the planner, the framework scans your entire project: existing components, code patterns, naming conventions, and your project's code standards. It checks for reusable code so it does not build something that already exists. If anything is unclear in your request, the framework asks you specific clarifying questions before producing the plan. You will never see a plan based on guesswork.

Once the plan is ready, you see a detailed breakdown: which files will be created or modified, the implementation approach, the SDLC phases that apply, and any open questions. Review the plan. If it looks good, approve it. If something needs to change, give feedback and the framework revises the plan until you are satisfied.

AFTER YOU APPROVE THE PLAN

Once you approve, the framework takes over and runs through the SDLC phases automatically. You do not need to invoke each phase manually. Here is what happens:

- The framework writes the code step by step, following your project's conventions and the approved plan. It compiles after each change and auto-fixes any build errors.
- Once implementation is complete, an automatic code review runs. You see a structured report with findings, severity levels, and specific fix recommendations. Fixes are applied and re-reviewed until clean.
- Unit tests are generated and executed. You see pass/fail results and coverage numbers.

- The code is deployed to your local environment. You see confirmation that the new code is active.
- If the task produces something visual (a page, a component), the framework validates the deployed output against the original requirements. You see a validation report.

Throughout this process, you see status updates at each phase. If anything needs your attention, the framework pauses and asks. Otherwise, it runs to completion.

OTHER COMMON WORKFLOWS

Not every task needs the full planning lifecycle. The framework provides focused commands for specific operations. Use these when you need a targeted action outside of the main /planner flow:

- **/review-code**: Get a code review on specific files or recent changes. Useful when you have written code manually or received changes from a colleague and want them checked against your project's standards before committing.
- **/debug-code**: Systematic debugging with root cause analysis. Describe the error or unexpected behavior, and the framework investigates, forms a hypothesis with evidence, and applies a minimal fix. It never guesses.
- **/generate-docs**: Create architecture documentation, ADRs, runbooks, or READMEs based on your actual codebase. The framework reads your project before writing, so the documentation reflects what the code actually does.
- **/deploy-local**: Build and deploy to your local environment. Useful when you want to deploy without going through the full SDLC. The framework compiles, deploys, and confirms the code is active.
- **/validate-implementation**: Validate a deployed page or API against acceptance criteria. Useful after manual changes or when you want to re-check a previously deployed component.

Each of these commands follows the same pattern: you invoke it, the framework does the work, and you review the output. They are all available in Cursor, VS Code with Copilot, and Claude Code using the invocation methods described above.

Chapter 4: Use Cases

This chapter walks you through 10 real workflows, step by step, from what you type to what you see. If this is your first time using the framework, start here.

1 Figma Design to Component

You have a Figma design for a hero banner and you want to turn it into a working, deployed component. Instead of manually extracting colors, fonts, and spacing from Figma, you let the framework handle everything.

YOU TYPE

```
/planner https://figma.com/design/abc123/Hero-Banner
```

WHAT HAPPENS NEXT

- **1. The framework reads your Figma design.** It connects to the Figma desktop app and extracts every detail: exact hex colors, font families and sizes, padding and margins, component variants (default, hover, active, disabled), responsive breakpoints, and image/icon asset URLs. You don't copy a single value manually.
- **2. Your codebase is scanned for reusable code.** Before building anything new, it searches for existing components that look similar. If you already have a banner component, the plan says "extend existing" instead of "build from scratch." This prevents duplicate code.
- **3. You see a plan and approve it.** The plan shows: files to create/modify, exact design values to use, approach (extend vs build new), and the SDLC flow. If something is wrong, you say so and the plan adjusts. Once approved, the framework takes over.
- **4. Code is written using YOUR design values.** The actual colors, fonts, and spacing from Figma, not generic placeholders. If Figma says #86BC25 with 12px padding, the code says exactly that. Compiles immediately. Build failures are auto-fixed.
- **5. Automatic code review.** A specialist reviewer checks against your project's standards: naming, accessibility (WCAG 2.2 AA), security, performance. You see a report with file:line, severity, and fix recommendations. Fixes are applied and re-reviewed until clean.
- **6. Tests generated and run.** Unit tests targeting 80% coverage. Failures are auto-fixed. "All tests passed. Proceeding to deploy."
- **7. Deploy.** Code deploys to your local instance. Logs checked, new code confirmed active.
- **8. Real content and assets.** The framework downloads actual images and icons from Figma, uploads to your asset store, and authors page content with those real assets via REST APIs. Not placeholders. The actual design assets.
- **9. Validated against the design.** Captures the Figma design and a screenshot of the live component, compares them side by side. You see a report: layout, colors, fonts, spacing. Pass/fail per check with a CSV report.

End result: One command, one URL. The framework built the component, reviewed it, tested it, deployed it, authored content with real assets, and validated it against the original design.

Skills auto-loaded: figma-context, planning-standards, frontend-code-standards

Agents invoked: planner, code-explorer, frontend-dev, code-reviewer, junits-specialist, validation-tester

SDLC phases: Plan > Build > Review > Test > Deploy > Author > Validate

You have a user story assigned to you in Azure DevOps or Jira. Instead of reading the acceptance criteria, manually figuring out what to build, and writing the code. You let the framework do the heavy lifting.

YOU TYPE

/planner #12345

For Jira, use the ticket key instead: /planner PROJECT-101

WHAT HAPPENS NEXT

- **1. The framework fetches the full story.** It connects to your work item tracker (Azure DevOps or Jira via MCP) and pulls everything: title, description, acceptance criteria, linked work items, priority, story points, and any attached files or URLs. You do not paste requirements. It reads them from the source. If the story has a linked Figma URL, the framework reads the design too.
- **2. Requirements become actionable specs.** The raw story data gets structured into something the framework can execute. "User should be able to filter by date" becomes a specific implementation spec: which component to build, which API to call, how to handle edge cases, what file paths to create. Each acceptance criterion becomes a testable condition that maps to the final implementation.
- **3. Your codebase is scanned for reuse.** Before building new code, the framework checks what already exists. You see things like: "Found existing DateFilter component. The plan will extend this rather than building new." This prevents duplicating work that's already been done.
- **4. You see a plan and approve it.** The plan maps every acceptance criterion to the implementation: what to build, where, how. You see file paths, the implementation approach, which patterns to follow, and any open questions. If the story is vague, the framework asks you to clarify before proceeding. It will not guess at requirements.
- **5. The full SDLC runs automatically.** Once you approve, the framework writes the code, compiles, reviews, generates tests, and deploys. All without further input. You see status updates at each phase: "Build passed," "Review complete: 2 issues found and fixed," "Tests passed," "Deploy successful."
- **6. Validation adapts to the task.** If the story produces a UI component, content is authored and visual validation runs. If it's a backend service, the framework validates via API calls or log checks. If it's a configuration change, it verifies the config is active. The framework adapts. Not every story needs every phase.
- **7. Everything traces back to the story.** At the end, every implementation decision maps to a specific acceptance criterion. Test cases reference requirements. The validation report shows which criteria pass. This traceability means you can prove the implementation matches the story.

Skills auto-loaded: story-context, planning-standards, figma-context (if Figma linked)

Agents invoked: planner, code-explorer, backend-dev/frontend-dev, code-reviewer, junits-specialist, validation-tester

SDLC phases: Plan > Build > Review > Test > Deploy > Author (if UI) > Validate

You see a component on a live website: a navigation bar, a card layout, a pricing table. You want to build something that looks like it in your project. You want the visual result, but built with YOUR code conventions.

YOU TYPE

/planner replicate the navigation bar from <https://example.com/homepage>

WHAT HAPPENS NEXT

- **1. The framework inspects the live page.** It opens the URL in Chrome (via DevTools MCP) and reads what the browser actually renders, not the site's source code, but the computed result: exact colors on screen, computed font sizes, CSS grid/flex layout, hover effects, responsive behavior at different screen widths. It also captures interaction patterns like animations and transitions. This is different from Figma. You are getting what actually renders in the browser, not what was designed.
- **2. Patterns are extracted and mapped.** The framework pulls out a structured set of patterns: HTML structure, typography scale, color palette, spacing system, interaction states. These become the visual reference. But here's the key: the plan uses the live site as a VISUAL reference while implementing using YOUR project's code patterns. If your project uses BEM naming and the live site uses utility classes, your component gets BEM names.
- **3. You approve the plan.** The plan shows how the live site's visual structure maps to your project's architecture. You see which existing components can be reused and what needs to be built new.
- **4. The automated SDLC runs.** Code is written to match the live site visually but follow your project's patterns. Build, review, test, deploy. Same flow as every other workflow. The reviewer checks against YOUR standards, not the live site's standards.
- **5. The result is compared to the original.** The framework captures the original live site component and your newly deployed component, then compares them visually. You see where they match and where they differ.

Skills auto-loaded: live-site-context, planning-standards, frontend-code-standards

Agents invoked: planner, code-explorer, frontend-dev, code-reviewer, junits-specialist, validation-tester

SDLC phases: Plan > Build > Review > Test > Deploy > Validate

Something is broken. A test is failing, there's a runtime error, a component doesn't render correctly, or the behavior doesn't match requirements. You need to find the root cause and fix it, not just make the error go away.

YOU HAVE TWO OPTIONS

OPTION A: PLAN FIRST, THEN FIX

`/planner fix the login form. Validation passes but form doesn't submit`

- **When to use this:** The bug isn't straightforward, it might span multiple files, or you want a structured investigation before changing anything.
- **What you see:** The framework scans the related code across your project, traces the execution path, and identifies potential root causes. It presents a fix plan: which files are involved, what the hypothesis is ("Form submission handler is bound to the wrong event on line 47"), and how the fix will be tested. You approve, and the framework implements the fix, reviews it, and runs tests to confirm.

OPTION B: DEBUG DIRECTLY

`/debug-code NullPointerException in UserService line 142`

- **When to use this:** You know what is failing: the error message, the file, maybe even the line. You want to go straight to fixing it without a full plan.
- **What you see:** The framework reads the failing code, traces the execution path, and forms a hypothesis with evidence: "This fails because the user object is null when the session expires. The null check on line 138 only covers the initial load path, not the session timeout path." It shows you the evidence, validates the hypothesis, then applies a minimal fix that addresses the root cause, not the symptom.

WHAT THE FRAMEWORK WILL NEVER DO

- It will never say "let's try this and see if it works." Every fix must have a hypothesis backed by evidence from the code.
- It will never add a null check without explaining why the null happens. Masking symptoms is explicitly forbidden. It must understand the root cause.
- It will never skip verification. After every fix: the code rebuilds, the reviewer checks the fix is minimal and addresses root cause, tests run to confirm the fix and catch regressions.

Bug fixes skip the Author and Validate phases. They are code-level changes, not renderable output that needs content or visual validation.

Option A agents: planner, code-explorer, backend-dev/frontend-dev, code-reviewer, junits-specialist

Option B agents: main agent directly, no subagents, no planner

SDLC phases: Plan (A only) > Build > Review > Test (no Author, no Validate)

You have written code, either manually or through the framework, and you want it reviewed before committing. Or you want to review a specific file or folder to check for issues.

YOU TYPE

`/review-code`

You can optionally target specific files: `/review-code src/components/Header`

WHAT HAPPENS NEXT

- **1. The reviewer reads your project's standards first.** Before looking at a single line of code, it loads the `code_standards` from `CLAUDE.md`. Your project's naming conventions, architecture patterns, error handling approach. This means the review is based on how YOUR project does things, not generic best practices from the internet.
- **2. Security scan (CRITICAL/HIGH).** The first pass is always security: the things that could cause real damage. It scans for: XSS vulnerabilities, SQL injection, hardcoded secrets or API keys in code, CSRF issues, insecure API patterns. These get flagged as CRITICAL or HIGH severity.
- **3. Logic and architecture review (HIGH/MEDIUM).** Next it examines the logic: off-by-one errors, null safety gaps, race conditions, N+1 database queries, missing error handling, violations of SOLID principles, performance anti-patterns. These are typically HIGH or MEDIUM.
- **4. Standards compliance (MEDIUM/LOW).** Finally, it checks conventions: does this match the project's naming patterns? Are imports structured correctly? Is the file in the right location? Is the error handling consistent with the rest of the project? These are MEDIUM or LOW.
- **5. You see a structured report.** Every finding includes: the exact file and line number, the severity level (CRITICAL/HIGH/MEDIUM/LOW), a specific description of what's wrong (not vague "consider refactoring"), and a specific recommendation for what to change. You see exactly what's wrong and exactly how to fix it.
- **6. Fixes are applied and re-reviewed.** The reviewer itself never touches your code. It only reports findings. The main agent reads the report, applies the recommended fixes, rebuilds the code, and sends it back for re-review. This cycle repeats until the review comes back clean.

Why the reviewer never fixes code itself: This is a deliberate design choice. When a reviewer also fixes, it has an incentive to minimize findings. Fewer findings means less work. By separating review from fix, the reviewer reports everything honestly, and a separate process fixes everything reported. Nothing gets swept under the rug.

Skills auto-loaded: frontend-code-standards (for .js, .css, .less, Handlebars, React, Angular, Vue files)

Agents invoked: code-reviewer (review and report only, never edits files)

Content Authoring and Publishing

Your component is deployed and running on your local instance. Now you need to add actual content: text, images, configuration. So you can see it working with real data. And optionally, push that content live.

AUTHORING CONTENT

YOU TYPE

```
/author-content add hero banner content to the homepage with the Figma assets
```

- **1. Assets are fetched from the design.** If the original component was built from a Figma design, the framework goes back to Figma, downloads the actual images, icons, and logos from the design file, and uploads them to your project's asset store on the running instance. Real assets, not placeholders.
- **2. Content is created via REST APIs.** The framework uses REST API calls to create or update content on the running local instance: creating content nodes, setting property values (text, images, links), configuring component dialogs, structuring page layout. It never edits your source code files to create content. Content lives on the running instance, not in your codebase. This is a critical rule.
- **3. The content is verified.** After authoring, the framework loads the page and confirms the component is rendering with the new content. If something didn't take, it fixes the content and re-checks.

PUBLISHING CONTENT

YOU TYPE

```
/publish-content homepage
```

- **1. You're asked to confirm.** Publishing pushes content to a shared instance that others may be using. The framework always asks for explicit confirmation before publishing. This is mandatory, not optional. You confirm by saying yes.
- **2. Content replicates.** The authored content is replicated from the author instance to the publisher instance. The framework verifies it's live and rendering correctly on the publisher.

WHEN TO USE THESE COMMANDS

- After deploying a new component and you want to see it with real content, not an empty shell
- After a Figma-to-component workflow, to populate the component with the actual design assets
- When updating existing content on a deployed page

If you ran the full SDLC from a Figma design, the Author phase happens automatically as part of the pipeline. you don't need to run `/author-content` separately. These commands are for when you want to author or publish outside of the main SDLC flow.

Agents invoked: None. Both are operational commands executed by the main agent directly

SDLC: Bypasses the planner. Operational command exception

You need documentation: an architecture overview for onboarding new team members, a decision record for a technical choice, a deployment runbook, or a README that actually reflects the current state of the code.

YOU TYPE

```
/generate-docs architecture overview of the project
```

Other examples: `/generate-docs ADR for choosing Redis over Memcached` | `/generate-docs deployment runbook` | `/generate-docs README for developer onboarding`

WHAT HAPPENS NEXT

- **1. The framework scans your entire codebase.** Before writing a single word, it reads your project: module structure, technology stack, dependencies, design patterns in use, configuration files, API endpoints, deployment setup. This is the evidence base. Every statement in the documentation will reference actual code.
- **2. The document is generated based on what you asked for:**
 - **Architecture Decision Record (ADR):** Captures WHY a decision was made: the context, alternatives considered, what was chosen, and the trade-offs. Follows the standard format. Useful when someone asks "why did we do it this way?" six months from now.
 - **Architecture Overview:** How the system is structured: modules, data flow, integration points, deployment topology. For someone who needs the big picture.
 - **Runbook:** Step-by-step operational guide based on your actual scripts, configurations, and deployment setup. Not a generic template. A guide built from your real infrastructure.
 - **README:** Developer onboarding: how to set up, run locally, run tests, deploy, and contribute. Written from your actual package.json, build scripts, and project structure.
- **3. Everything is evidence-based.** File paths, class names, configuration values, API routes. All pulled from your real codebase. The documentation describes what the code actually does, not what someone remembers it doing.

Key difference from writing docs manually: The framework reads the entire codebase before writing. It eliminates the most common documentation problem: being out of date the moment it's written. When the code changes, regenerate the docs.

Agent invoked: docs-scribe (scans codebase, generates documentation)

Skills loaded: None. docs-scribe works from codebase analysis, not domain skills

SDLC: Bypasses the planner. Documentation is a framework maintenance exception

You are facing a technical decision: should you use server-side rendering or client-side rendering? Should you build a custom solution or extend an existing library? Should you use a monolith or microservices? Instead of debating in meetings or guessing, you let the framework research, analyze, and deliver a single definitive recommendation backed by evidence from your codebase and platform documentation.

YOU TYPE

`/planner evaluate whether to use SSR or CSR for the product catalog`

Other examples: `/planner should we use Redis or in-memory caching for session storage` | `/planner evaluate GraphQL vs REST for the content API`

WHAT HAPPENS NEXT

- **1. The framework scans your codebase for context.** Before evaluating anything, it reads your project: what technologies are already in use, what patterns the team follows, what infrastructure is in place. This sets the hard constraints. If your project already uses a specific rendering approach or caching layer, the recommendation accounts for that.
- **2. Platform documentation is researched.** The framework fetches current documentation for the technologies being evaluated. Not outdated blog posts or general opinions, but the official docs for your specific platform version. This ensures the recommendation is based on what actually works, not what worked two years ago.
- **3. Constraints are separated into hard and flexible.** Hard constraints are non-negotiable: what your architecture enforces, what your platform supports, what your code standards require. Flexible constraints are trade-offs: performance targets, team familiarity, timeline pressure. You see both clearly.
- **4. You get ONE recommendation, not a list of options.** The framework does not present "Option A vs Option B" and leave you to decide. It picks the best approach and defends it. You see: what was chosen, why it was chosen, what alternatives were considered, and why they were rejected. Every claim is backed by evidence from your codebase or the documentation.
- **5. Trade-offs are explicit.** No recommendation is perfect. The framework tells you exactly what you gain and what you give up. If SSR is recommended, it explains the trade-offs: better SEO and initial load, but more server load and complexity. You make the final call with full visibility.

Key difference from a typical architecture discussion: The framework bases its recommendation on your actual codebase and current platform documentation, not on general best practices. Two teams with different codebases asking the same question may get different recommendations, because the answer depends on what already exists in each project.

Agents invoked: solutions-architect (delivers recommendation), code-explorer, research-intelligence

Skills auto-loaded: planning-standards (workflow-design for architectural decisions)

Output: Single recommendation with trade-off analysis, not a comparison table

You have a feature request, a business need, or a vague idea that needs to be turned into structured, actionable requirements before anyone writes code. Instead of manually reading through stories, interviewing stakeholders, and writing requirement documents from scratch, you let the framework analyze the input, scan your codebase for context, and produce a structured requirements document.

YOU TYPE

`/planner analyze requirements for the user preferences feature from #12345`

Other examples: `/planner gather requirements for adding multi-language support` | `/planner analyze NFRs for the checkout flow performance`

WHAT HAPPENS NEXT

- **1. The framework classifies the analysis type.** Based on your request, it determines whether this is a discovery (scope definition), requirements extraction (functional and non-functional), NFR analysis (performance, security, scalability targets), or a feasibility assessment (can we build this, and what are the risks). Each type follows a different workflow.
- **2. If a story or ticket is referenced, it is fetched automatically.** The framework connects to your work item tracker (Azure DevOps or Jira) and pulls the full story: title, description, acceptance criteria, linked items, attachments, and any Figma URLs. You do not paste requirements manually.
- **3. Your codebase is scanned for existing capabilities.** Before defining what to build, the framework checks what already exists. If you ask for multi-language support and there is already a partial implementation, the requirements account for that. This prevents requirements that duplicate existing functionality.
- **4. Hidden requirements are surfaced.** The framework looks for implicit requirements that the original story did not state explicitly. If the story says "user can save preferences" but does not mention validation, error handling, or data format, the framework flags these as gaps and asks you to confirm.
- **5. You get a structured requirements document.** The output includes: clearly defined functional requirements, non-functional requirements with measurable targets (not vague "should be fast" but specific thresholds), acceptance criteria mapped to testable conditions, identified dependencies and risks, and open questions that need stakeholder input.
- **6. Open questions are presented for your input.** The framework does not fill in gaps with assumptions. If something is unclear or ambiguous, it asks you specific questions. You answer, and the requirements document updates accordingly.

Why this matters: Requirements gathering is where most projects go wrong. Vague requirements lead to wrong implementations, which lead to rework. By structuring requirements upfront with codebase awareness, the framework catches gaps before any code is written.

Agents invoked: planner (analysis workflow), code-explorer, research-intelligence

Skills auto-loaded: planning-standards (workflow-analysis), story-context (if ticket referenced)

Output: Structured requirements document with acceptance criteria and open questions

You have an idea but you are not ready to commit to an approach yet. Maybe you want to add real-time notifications, rethink the navigation structure, or modernize a legacy module. You need the framework to help you think it through: what exists today, what the options are, what fits your project best, and what the effort looks like. This is exploration, not implementation.

YOU TYPE

`/planner explore approaches for adding real-time notifications to the dashboard`

Other examples: `/planner explore how to modernize the legacy search module` | `/planner what would it take to add offline support to the mobile view`

WHAT HAPPENS NEXT

- **1. The framework researches your idea against your codebase.** It scans your project to understand what exists today: current architecture, related components, integration points, and any prior work in the area. If you want real-time notifications and there is already a WebSocket utility buried in the codebase, the exploration surfaces it.
- **2. Platform capabilities and limitations are researched.** The framework fetches current documentation to understand what your platform supports natively, what requires third-party tools, and what the known limitations are. This grounds the exploration in reality rather than theoretical possibilities.
- **3. The architect evaluates and recommends.** Based on codebase findings and platform research, the solutions architect delivers a recommendation: the best approach for YOUR project, considering your existing patterns, team conventions, and technical constraints. You see what would need to change, what can be reused, and where the complexity lies.
- **4. Effort and impact are estimated.** The exploration includes a complexity assessment (S/M/L), the files and modules that would be affected, dependencies that need to be considered, and a rough scope of what building this would involve. This helps you decide whether to move forward.
- **5. You decide the next step.** After reviewing the exploration, you can: approve and move directly into planning (`/planner` will produce a full implementation contract), ask for deeper analysis on a specific aspect, pivot to a different approach based on what you learned, or shelve the idea for later with the exploration saved as a reference.

How this differs from architectural decision making: Architectural decisions start with a known question (SSR vs CSR, Redis vs Memcached). Solution exploration starts with an open-ended idea ("I want to add notifications") and helps you figure out what the right questions even are. It is the step before you have a concrete decision to make.

Agents invoked: planner, solutions-architect, code-explorer, research-intelligence

Skills auto-loaded: planning-standards (workflow-analysis or workflow-design based on scope)

Output: Exploration report with recommendation, effort estimate, and next-step options

Chapter 5: Customization

The framework is designed to be adapted to your project. Everything (code standards, skills, agents, commands, hooks, rules, MCP servers) can be modified, added, or removed. This chapter covers how.

Built-in help: The framework includes a utility skill called framework-guidance.

What it provides: Design principles, anti-patterns, modification workflows, dos and don'ts.

How to use it: It loads automatically when you modify framework components.

MODIFY CODE STANDARDS

After running /initialize-setup, open the **code_standards** section in CLAUDE.md. This is where your project's coding conventions live. Every agent reads this section before writing or reviewing code. Add, modify, or remove rules as needed. Use the MUST/MUST NOT/SHOULD format:

- **MUST:** strict rules that agents will always follow (e.g., "MUST use camelCase for variable names")
- **MUST NOT:** prohibitions (e.g., "MUST NOT use inline styles")
- **SHOULD:** recommendations that agents follow unless there's a good reason not to

These are baseline standards from the scan. Review them, tighten what matters to your team, remove what doesn't apply. The framework adapts to whatever you define here.

CREATE OR MODIFY SKILLS

Skills are the framework's domain knowledge modules. To add a new skill:

- Create a folder under **.claude/skills/** with a kebab-case name (e.g., my-new-skill)
- Add a **SKILL.md** file with YAML frontmatter (name, description, triggers) and instructions
- Optionally add a **references/** subfolder for detailed documentation loaded on demand

Use the **skill-creator** utility skill for guided creation. It walks you through the structure, validates the YAML, and ensures the skill follows framework conventions. To review an existing skill for accuracy and consistency, use **skill-review**. It audits documentation, checks for outdated patterns, and suggests fixes.

CREATE OR MODIFY AGENTS AND COMMANDS

- Agents live in **.claude/agents/** as markdown files with YAML frontmatter (name, description, handoffs, hooks)
- Commands live in **.claude/commands/** as markdown files. Thin wrappers that invoke agents with context
- The **framework-guidance** skill has detailed reference docs on agent structure, handoff patterns, Stop hook design, and command conventions

When creating new agents or commands, the framework-guidance skill loads automatically and provides the design principles, anti-patterns to avoid, and validation checklists to follow.

MODIFY HOOKS

Hooks are defined in **.claude/settings.json**. The framework ships with three: PreCompact (preserves context during compaction), Stop (ensures SDLC continuity), and SubagentStart (injects task tracking into subagents). You can modify these or add new ones. The framework-guidance skill documents hook types, trigger points, and best practices.

MODIFY SETTINGS AND PERMISSIONS

.claude/settings.json controls which commands are auto-allowed, which require confirmation (ask rules), and which are blocked (deny rules). It also defines allowed WebFetch domains and hook configurations. Review the default

settings and adjust for your project's security requirements.

ADD MCP SERVERS

- **Cursor:** Add to `.cursor/mcp.json`
- **GitHub Copilot:** Add to `.vscode/mcp.json`
- **Claude Code:** Add to `.mcp.json` at project root

Keep the total MCP server count manageable. Each server adds tools to the AI context window. Too many servers increase token usage and can reduce response quality.

ADD PROJECT RULES

Place rule files in `.claude/rules/` (Claude Code + Copilot) or `.cursor/rules/` (Cursor only). These load alongside `CLAUDE.md` and enforce project-specific constraints. Use `MUST/MUST NOT/SHOULD` format. Rules are useful for team-specific conventions that go beyond `code_standards`: deployment policies, branch naming, PR templates, forbidden patterns.

Chapter 6: FAQs

GETTING STARTED

What is CLAUDE.md? It is the prime orchestrator file. It is loaded at the start of every conversation across all three tools (Cursor, Copilot, Claude Code). It contains your project context, code standards, SDLC phases, delegation rules, and engineering principles. Every agent reads it before acting. It's what makes the framework behavior consistent across developers and sessions.

Do I need all three tools (Cursor, Copilot, Claude Code)? No. You only need one. The framework works identically across all three. Pick whichever your team uses. The framework files (.claude/, CLAUDE.md) are the same regardless of the tool.

What happens if I skip /initialize-setup? The framework still works. When you start your first task, the agent detects the empty project_context and handles the setup dynamically. Running /initialize-setup is recommended because it gives the framework a head start, but it is not a blocker.

How do I know which slash command to use? Start with /planner for any development task. The planner figures out the rest: which agents to invoke, which SDLC phases apply, what complexity level it is. The other commands (/review-code, /debug-code, /author-content, etc.) are for specific operations outside the main planning flow.

Do I need to set up all MCP servers? No. MCP servers are optional. The framework works without any MCP servers. You just lose the ability to read Figma designs, fetch stories from ADO/Jira, and inspect live sites directly. You can always provide that context manually in your prompt instead.

PLANNING & SDLC

Can I skip the plan and go straight to building? For development tasks, no. The framework requires a plan. This is intentional. The plan catches issues before code is written: missing requirements, duplicate components, wrong architecture. For operational tasks (debugging, deploying, authoring content), you can use commands directly.

What if I disagree with the plan? Tell the framework what's wrong. It will adjust the plan based on your feedback. You can modify file paths, change the approach, add requirements, or reject sections entirely. The plan is a proposal. You have final approval.

What does S/M/L complexity mean? S (small) is a quick change: single file, clear scope. M (medium) is a feature spanning multiple files but within one domain. L (large) is a significant feature crossing multiple domains. S/M tasks run fully automatically. L tasks pause before deploy for your confirmation and may use subagents for parallel work.

Why does the reviewer never fix code? Separation of concerns. If the reviewer also fixed code, it would have an incentive to minimize findings (fewer findings = less work). By keeping review and fix separate, the reviewer reports honestly, and a separate process fixes everything. This produces better code quality.

What if the build fails? The framework reads the error, forms a hypothesis about the cause, and fixes it automatically. It then rebuilds. If it fails again, it tries a different approach. You see status updates throughout. If it can't resolve the issue after multiple attempts, it asks for your help.

SKILLS & AGENTS

What are skills exactly? Skills are domain knowledge modules that agents load on demand. They contain platform-specific patterns, conventions, and reference documentation. An agent working on frontend code loads frontend-code-standards. An agent processing a Figma URL loads figma-context. Once the task completes, the skill context is discarded. This keeps token costs low. Only the right knowledge at the right time.

How do I create a new skill? Use the **skill-creator** utility skill. It walks you through the process: creating the folder structure, writing the YAML frontmatter, defining triggers, and adding reference documents. You can also create one manually. Create a folder under `.claude/skills/` with a `SKILL.md` file.

Can I add my own agents? Yes. Create a markdown file in `.claude/agents/` with YAML frontmatter (name, description, handoffs, hooks) and a role definition in the body. The framework-guidance skill has templates and best practices for agent design.

What's the difference between a command and an agent? Commands are thin wrappers. A command file invokes one or more agents with pre-configured context. Think of commands as shortcuts and agents as the actual specialists. `/review-code` is the command; `code-reviewer` is the agent it invokes.

MCP & INTEGRATIONS

What if I use a different design tool (not Figma)? You can still use the framework. Describe the design in your prompt. Provide colors, fonts, spacing, layout descriptions manually. Or create a custom skill that integrates with your design tool if it has an API or MCP server.

Can I add more MCP servers? Yes. Add them to the appropriate config file (`.cursor/mcp.json`, `.vscode/mcp.json`, or `.mcp.json`). Keep the total count manageable. Each server adds tools to the AI context window. Too many servers increase token usage and can reduce response quality.

What if I use GitHub Issues instead of ADO/Jira? The story-context skill currently supports Azure DevOps and Jira. For other trackers, you can paste the issue content directly into your prompt, or create a custom skill that integrates with your tracker's API.

USE CASES

What if I don't have Figma Dev Mode? You need a Dev or Full seat on a paid Figma plan. Without it, the MCP server cannot connect. You can still describe the design in your prompt and provide values manually.

Can I combine Figma with a user story? Yes. Use `/planner #12345 https://figma.com/...` and the framework merges acceptance criteria from the story with design tokens from Figma into one plan.

What if the story is vague or incomplete? The planner flags open questions and asks you to clarify before proceeding. It presents specific questions based on what is missing, not generic "please clarify" prompts.

Can I add context beyond the story? Yes. Add notes after the ID: `/planner #12345 use the existing sidebar pattern`, not a new layout. The framework combines the story context with your instructions.

How is live site replication different from Figma? Figma gives you the intended design (design tokens as specified by the designer). Live site gives you the actual rendering (computed styles as the browser calculates them). Live site extraction also includes interaction patterns and responsive behavior that Figma designs may not fully specify.

Does live site replication work with authenticated pages? Yes. If you are already logged into the site in Chrome with remote debugging enabled, the framework connects to that browser session. No separate login automation is needed.

When should I use `/planner` vs `/debug-code` for bugs? If the bug is in one file and you know where, use `/debug-code`. It is faster. If the bug could be anywhere, involves multiple components, or you are not sure what is causing it, use `/planner`. It investigates systematically before touching any code.

Can I disagree with a review finding? Yes. If the reviewer flags something you intentionally wrote that way, tell it why. The framework respects your judgment and will not blindly apply fixes you disagree with.

Does the review happen automatically during the SDLC? Yes. When you build a component through `/planner`, the review phase runs automatically after the code is written. `/review-code` is for when you want to review code outside of that automated flow, like code you wrote manually or changes from a colleague.

Why can't I just edit content files directly? Editing source files puts content in your codebase, which gets committed and deployed to every environment. Authored content should live on the running instance, managed through the CMS, not checked into version control.

When should I regenerate documentation? After any significant code change: new modules, changed APIs, updated deployment process. The docs are only as current as the last time they were generated.

Can I edit the generated docs? Yes, and you should. The framework generates a starting point based on the code. Add the context only you know: business decisions, team conventions, historical context, lessons learned. The generated doc is a foundation, not the final version.

ADVANCED

How does context compaction work? When the AI's context window fills up, it compresses the conversation. The `PreCompact` hook fires and injects a structured summary: what was done, what is in progress, what to do next, and the active plan. This ensures the agent picks up exactly where it left off after compression. No lost context.

What are hooks and why do they matter? Hooks are automated rules that fire at lifecycle points: before compaction, when an agent finishes, when a subagent starts. Unlike instructions that guide behavior, hooks enforce behavior. An agent can ignore an instruction; it cannot bypass a hook. They're what makes the SDLC flow reliably from start to finish without manual intervention.

How does the framework handle large tasks differently? For S/M complexity, the main agent does everything directly, no subagents. For L complexity, the framework may use subagents for parallel work (e.g., backend and frontend simultaneously). But every subagent handoff includes the actual design values and patterns inline, not "refer to the plan" but the actual extracted values. This prevents context loss across agent boundaries.

Can I use this framework for non-CMS projects? Yes. The framework is tool-agnostic and stack-agnostic. The core (agents, commands, SDLC, hooks) works on any project. The CMS-specific and AEM-specific skills are what you replace when adapting to a different stack. The utility skills (framework-guidance, skill-creator, skill-review, initialize-setup) work on any technology.

How do I adapt the framework to a different tech stack? Run `/initialize-setup` on your project — it detects the stack automatically. Then replace the platform-specific skills (e.g., `aem-code-standards`) with skills for your stack. The utility skills, agents, and commands are generic and work as-is. Use `skill-creator` to build new stack-specific skills.